HW2 Problem Set

ECS 240

Due: Friday, November 20

This problem set contains 5 problems. Problems 1 and 5 are pure theory problems, and 2, 3, and 4 are coding problems. Most problems have multiple parts. For the coding problems, edit and add your source code in the provided `src/main.rs`. Solutions to the theory and free response questions should be written in LaTeX.

---

1. In class (Lecture 2, Part 1) we discussed that for compilers, there are three different languages involved – each with their own semantics, and a valid compiler must be consistent with all three of them. Namely:

   - The meta language M (language in which we write the compiler)
   - The source language S (language in which we write the program)
   - The target language T (language to which we compile the program).

   This problem (in part (a)) asks you to complete the definition of a *correct* compiler (one which implements the semantics), based on these three semantics. Then you will apply your definition to parts (b)-(d).

   (a) Complete the definition below based on the template that we started in class. **Hint:** you may find it helpful to reference the definition of when an interpreter satisfies a semantics that we saw in class in Lecture 1, Part 4. Your definition should follow a similar idea.

   Definition 1. *Let M be the semantics (set of $(\texttt{INPUT}, \texttt{prog}, \texttt{OUTPUT})$) for the metalanguage. Let S be the semantics of the source language. Let T be the semantics of the target language.*

   A compiler C is a program in the metalanguage M.

   *A compiler C **is correct**, or **satisfies** the semantics from S to T, if: ...*

   TODO: Complete the definition here.

   (b) Define the "do nothing" compiler from S to S to be a compiler which simply takes the input program, and returns it without any changes. Formally, this is a program in the metalanguage M such that

   $$(\texttt{INPUT}, \texttt{do_nothing}, \texttt{OUTPUT})$$

   is valid if and only if $\texttt{INPUT} = \texttt{OUTPUT}$.

Use your definition from part (a) to prove or disprove the following: Regardless of the semantics of S, and even if S is nondeterministic, a do-nothing compiler is a valid compiler from S to itself. Check that your result agrees with your intuition and make any edits to your definition if needed.

(c) Use your definition from part (a) to prove the following: If C_1 is a correct compiler from S_1 to S_2, and C_2 is a correct compiler from S_2 to S_3, then the compiler C defined by the following pseudocode is a correct compiler from S_1 to S_3:

```
Compiler C from S1 to S3:
On input prog0:
- Run C1: set prog1 = C1(prog0).
- Run C2 on the result: set prog2 = C2(prog1).
- Return prog2.
```

You may assume that the meta-language M has some program for the pseudocode above, and that its semantics works how you would expect. I.e., it runs C_1, followed by C_2, and returns the result, similar to how `Seq` works for `RandProg`.

Note: This part is essentially showing that compiling a program through an intermediate representation works out. In this case, the intermediate representation is S_2 and the overall compiler is from S_1 to S_3.

(d) Use your definition from part (a), together with the definition of a when an interpreter satisfies the semantics from class, to prove the following result: If C is a correct compiler from S to T, and I_T is an interpreter for T that implements the semantics, then running C, followed by running I_T on the result, results in a correct interpreter I_S for S. That is, the following pseudocode is a valid interpreter:

```
Interpreter I_S for S:
On input (INPUT, prog):
- Run C: set prog' = C(prog).
- Run I_T((INPUT, prog')).
```

As with part (c), you may assume that the meta-language M has some program for the pseudocode above, and that its semantics works how you would expect. I.e., it runs C, followed by I_T, and produces any resulting output.

(e) In your opinion, is the interpreter from (d) very useful? Why or why not?

2. This problem will ask you to expand the basic `RandProg` compiler from class to add some syntactic sugar. First, copy in any necessary code involving `RandProg` from the Lecture 2, Part 1 source code. Then complete the following (this was an exercise that we skipped in class):

(a) First, fix our compiler from class to produce directly executable Rust code. To do this, you will need to wrap the code in a `main` function, and you may also need to inline (or include with the compiled target code) one or more utility functions we used from `util.rs`. The compiler should now produce a single `String` for a `main.rs` source file, which can be run directly.

Write a unit test to see that it is working. Your unit test can simply check that the output is a String equal to correctly executable Rust code. However, you may also want to plug this into a `main.rs` file or put it on the Rust playground to see that it actually runs.

(b) Add syntactic sugar to our language for 4-bit integers, which support equality, less-than, and addition operations. Adapt and structure the compiler in such a way that the 4-bit integer logic is compiled out, and the 4-bit integers are reduced to Booleans. (Each 4-bit integer variable should be compiled as four separate Boolean variables.) See below for the function and type names required.

To do this correctly and without a lot of boilerplate code everywhere, you should see what additional methods or helper functions you can define to help out! See the **Helpful note:** in the Appendix included at the end of this PDF. Building on this note, you could write other methods or helper functions for some of the other common operations.

(c) Add at least two unit tests to ensure that your new compiler is working.

Function and type names: For part (a), use the same `RandProg` and compiler from class. For part (b), please define a Rust enum `RandProgP2` for the syntax with the syntactic sugar, `struct RandProgP2Compiler` for the compiler, which supports methods

```
impl RandProgP2Compiler {
    pub fn new() -> Self
    pub fn compile(&mut self, prog: &RandProgP2) -> String
}
```

For the unit tests in (a) and (c), please name these `test_2a`, `test_2c_1`, and `test_2c_2`. (You may add other unit tests with other names.)

3. This problem will ask you to expand the basic `RandProg` to add something resembling platform-specific code. Use the same `RandProg` enum from Problem 2. Then, complete the following (this was an exercise that we skipped in class):

(a) Add some "platform-specific code" for random number generation by allowing our language to be configured to different pseudorandom number generators at the back-end: (1) Rust thread RNG (default Rust random generator) (2) Always return Heads (3) Always return Tails (4) On Unix systems, use Unix's `/dev/random` for RNG rather than the built in Rust thread RNG.

(b) Add at least three unit tests to ensure that your new compiler is working.

(c) Then answer the following (in your LaTeX document and PDF): Why might it be more helpful to generate this platform-specific code, *after* any compiler analysis and optimization passes on various intermediate representations have completed? Comment on why you think this separated design might be commonly used in compiler development in practice.

Function and type names: Use `RandProg` for the syntax, and `RandProgP3Compiler` for the compiler. To get this to work, you will need to add a `RandProgRngConfig` enum to your compiler that tells which platform to compile for:

```
#[derive(Debug, Copy, Clone)]
pub enum RandProgRngConfig {
    DefaultRng,
```

```
    AlwaysHeads,
    AlwaysTails,
    DevRandom,
}
```

Then, your compiler should have two different constructors:

```
impl RandProgP3Compiler {
    pub fn new() -> Self // Use default Rng
    pub fn new_with_rng(config: RandProgRngConfig) -> Self
    pub fn compile(&mut self, prog: &RandProg) -> String
}
```

Please name your unit tests `test_3b_1`, `test_3b_2`, and `test_3b_3`. (You may add other unit tests with other names.)

4. This problem will revisit the BF language from HW1.

   One extremely important intermediate representation in the context of real compilers is that of a *control flow graph*. To define a control-flow graph for a language, we define a set of nodes and edges. These are defined as follows:

   - **Nodes:** The nodes are the set of **basic blocks** of the program. A basic block is a sequence of instructions, none of which is a branching or loop statement (with the possible exception of the very last statement). In the context of BF, that means it will be a sequence of instructions in the program that doesn't have the `[]` characters, except (possibly) its very last character. The idea of a "basic block" is that it is a basic sequence of instructions that can be executed in order – and is always executed in order, "as a whole" – with exactly one entry and exit point. In particular, it is impossible for any other part of the program to ever jump or branch to the middle of a basic block.

     Also, one special basic block is designated as the entrypoint of the program, and one as the exit point. The exit point block (unlike the others) may be empty, in the case that the program ends in a jump or branch statement (to distinguish from the block of instructions prior to the jump, and after the jump when the program will terminate). The entry point block should always be nonempty, unless the program itself is empty, in which case it should have only one basic block, which is both the entry and exit point.
   - **Edges:** The edges are then from one basic block to another, if it is possible for that basic block to transition to the other one after it finishes executing. For example, in languages with `if` statements, for the basic block immediately prior to an `if` statement, it would have an edge to both branches of the `if`. In languages with `loop`s (of all kinds), the basic block immediately prior to the loop will have an edge to both the block inside the loop, and after the loop. Often, these edges are labeled with `true` or `false` corresponding to the result of the prior test resulting in the branch.

   For this problem, write two functions:

   ```
   pub fn num_basic_blocks(bf_prog: &str) -> usize
   ```

which takes a BF program as a plain ASCII string and calculates the number of basic blocks; and

```
pub fn num_edges(bf_prog: &str) -> usize
```

which takes a BF program as a plain ASCII string and calculates the number of edges. Both functions may assume the BF program is valid (has matching loop instructions) It may be the empty string.

You do not need to actually write a compiler which takes the BF program and calculates its control flow graph as an intermediate representation as part of this problem, though you may choose to do so. We will grade only your two functions for correctness.

You may want to write some unit tests for your functions to ensure they are working correctly.

5. **Warning: This problem is difficult! :-)** To make it more accessible, I split it up into multiple parts, and some parts ask only for your informal reasoning rather than proofs. Please do your best to work through it and especially to understand the concepts involved. Come to office hours if you get stuck.

   Most programming languages that are designed and studied in practice are what is called *Turing-complete*. This can be defined as "has the ability to simulate, and can be simulated by, a Turing machine", or it may be defined more informally as simply "has the ability to simulate, and can be simulated by, any regular program" (such as one written in Python or C). Alternatively, it may be defined in terms of concepts introduced in HW1, as: the semantics of the language can both simulate BF, and can be simulated by BF.

   However, not all languages are Turing-complete. For example, from class, `RandProg` is not Turing-complete. It has only Boolean variables, so cannot store and access infinite memory. It would be Turing-complete if we were to add integer variables and loops (for example, `while` loops or `label/goto` statements).

   In the following there are six parts: Parts (a)-(b) will show that we can design a language that is *weaker* than Turing-complete, and parts (c)-(d), *stronger* than Turing complete. Parts (a) and (c) will ask for proofs, but parts (b) and (d) will only ask for your informal reasoning. Part (e) is a free response question, and part (f) is optional extra credit.

   For all parts, we could use BF is used as a stand-in language for a Turing-complete language. However, we will instead work with the following less-contrived language, called WHILE, which is closer to real programs. It has the following **Syntax** and **Semantics**:

   - **Syntax:**

     ```
     Var ::= x | y | z
     IntExpr ::= 0 | 1 | Var
                 | (IntExpr + IntExpr) | (IntExpr * IntExpr)
     BoolExpr ::= IntExpr < IntExpr | IntExpr == IntExpr
     Prog ::= Var := IntExpr
              | Prog ; Prog
     ```

```
        | If { BoolExpr } Then { Prog } Else { Prog }
        | While { BoolExpr } Do { Prog }
```

- **Semantics:** For the semantics, assume that we are given a deterministic reference interpreter for `WHILE` whose behavior is as you expect (for example, matching how while loops would be executed in Python or C), and that its output is taken as the definition. This is similar to how we saw in HW1 that `BF` semantics could be defined using a reference interpreter, for the non-extra-credit version.

  For the reference interpreter, assume that it takes the initial values of `x`, `y`, and `z` as input, and the final values of `x`, `y`, and `z` are output. Then, assume that $((x_0, y_0, z_0), \texttt{prog}, (x_1, y_1, z_1))$ is valid if, when running the program `prog` as you would expect (for example, interpreting If statements and While loops as they are usually understood), starting with initial values of the variables (x_0, y_0, z_0), (x_1, y_1, z_1) are the final values of the variables. The expression `While { BoolExpr } Do { Prog }` means that we check whether `BoolExpr` is true, and each time that it is true, we execute the loop body.

  Finally, there is one important edge case – what is the semantics when the WHILE program does not terminate? Assume in this case that the output is $(-1, -1, -1)$.[1]

To prove that one language can simulate another, you should do so by defining a function f, as in HW Part 1, that translates one language to another. It can be any mathematical function. Argue informally that your translation is correct based on the semantics of the source and target languages. The parts that show a simulation is *not* possible will ask only for an informal argument; for these parts, you should argue informally why no such function f can exist.[2]

(a) Consider the following language, similar to `WHILE`, called `FORLOOP`:

- **Syntax:**

```
Var ::= x | y | z
IntExpr ::= 0 | 1 | Var
          | (IntExpr + IntExpr) | (IntExpr * IntExpr)
BoolExpr ::= IntExpr < IntExpr | IntExpr == IntExpr
Prog ::= Var := IntExpr
       | Prog ; Prog
       | If BoolExpr Then { Prog } Else { Prog }
       | For { Var } From { IntExpr } To { IntExpr } Do { Prog }
```

- **Semantics:** The semantics is the same as for `WHILE`, except for `For` loops: in that case, the expression `For { Var } From { IntExpr } To { IntExpr } Do { Prog }` means that the starting integer i and ending integer j will be calculated up front, and then the for loop will be executed by setting `Var` to every value $i, i+1, i+2, \ldots, j-1$ (like a Python `range`). This remains the case even if the variable `Var` is modified in the loop body. So, modifications

[1] You might think that the output should be empty – i.e., the empty set of possible outputs – in this case. But this is undesirable! Recall that making the semantics the empty set (even on some particular input) leads to there being no possible valid interpreter for the language. See poll on Oct 28.

[2] There is a bit of a deficiency in our definition of a simulation function f, which is that the function f should really be required to be *computable* in some meta-language. This deficiency is explored in part (f), which is optional extra credit.

to `Var` in the loop body are only retained for the duration of that loop body, and don't affect the following loop iteration. If $j \leq i$ immediately before the start of an iteration, the for loop will terminate immediately.

`FORLOOP` is not Turing complete, but it is more specifically *weaker* than Turing complete: it can be simulated by `WHILE` programs, but not vice versa.

Prove the first direction, that `FORLOOP` can be simulated by `WHILE`.

(b) Argue informally in the other direction: why `WHILE` cannot be simulated by `FORLOOP`.[3]

(c) Now consider the following language, called `HALT`:

- **Syntax:**

```
Var ::= x | y | z
IntExpr ::= 0 | 1 | Var
            | (IntExpr + IntExpr) | (IntExpr * IntExpr)
BoolExpr ::= IntExpr < IntExpr | IntExpr == IntExpr
             | Halts { IntExpr }
Prog ::= Var := IntExpr
         | Prog ; Prog
         | If BoolExpr Then { Prog } Else { Prog }
         | While { BoolExpr } do { Prog }
```

Semantics: The semantics is again the same as for `WHILE`, except for `Halts` expressions. The boolean expression `Halts { IntExpr }` returns `True` if and only if the integer expression given to it - when converted to binary and then interpreted as an ASCII string – encodes a `WHILE` program that halts on input $(0, 0, 0)$.

`HALT` is not Turing complete, but it is more specifically *stronger* than Turing complete: it can simulate `WHILE`, but not vice versa.

Prove the first direction, namely that `HALT` can simulate `WHILE`.

(d) Argue informally in the other direction: why `WHILE` cannot simulate `HALT`.[4]

(e) Explain why, due to the above constraints, it will be impossible to write a correct compiler in *any* metalanguage – even one that is more powerful than Turing machines – from `HALT` to `WHILE` or from `WHILE` to `FORLOOP`.

(f) **(Optional – extra credit)** If, in the definition of HALT, we were to define `Halts { WhileProg }`, where `WhileProg` is simply a WHILE program – instead of `Halts { IntExpr }` with an integer whose binary expansion encodes a WHILE program – then HALT would not actually be strictly stronger than WHILE in the above sense. Why not?

Hint: this is due to a deficiency in our working definition of a simulation function, since we did not require f to be actually computable in the translation.

[3] There is one "surface-level" reason why this is the case, and one "deeper" reason. If you are not sure, don't worry about this comment. To really explore the deeper reason, we would have to change the semantics for nonterminating programs, for example to output $(0, 0, 0)$ in stead of $(-1, -1, -1)$.

[4] Make sure you don't assume that the function f is a real program! It could be any mathematical function.

Submission instructions:

- Upload your solutions (as a PDF) and your code (as a `.zip` file, as with HW0: including the entire repository directory including config files like `Cargo.toml`, but excluding `target/`) in Gradescope.
- Your PDF should be a single file `hw2.pdf`. If you use this LaTeX template to create your solutions, please remove the problem statements and include only your solutions.
- The `main.rs` should contain your solutions to all coding parts (including functions and types as described above). Please do not modify function signatures. You are welcome to add additional functions and tests – if you do so, please include all additional tests only at the bottom of the file.
- You may add utility modules, like `mod util;` then `use util::...`, if you feel that you need them; but you should put all the main functions with the names as above in `main.rs`. Please double check on Piazza before importing any external crates in your `Cargo.toml`.
- Make sure that `cargo build`, `cargo clippy`, `cargo test`, and `cargo fmt` (or `cargo fmt --check`) pass without any errors.

Appendix: Helpful note on reducing boilerplate for `RandProg`:

In class, we saw that creating `RandProg` programs via our Rust enum directly produces a lot of `Box::new(...)` and `.clone()` syntax. This is neither necessary nor desirable! Here is a helpful hint about how to avoid that. You can create an impl block like this, and give it various shortcut methods for creating various `RandProg` programs. For example:

```
impl RandProgP2 {
    pub fn seq(prog1: &RandProgP2, prog2: &RandProgP2) -> RandProgP2 {
        RandProgP2::Seq(
            Box::new(prog1.clone()),
            Box::new(prog2.clone())
        )
    }
    pub fn if_then(/* fill in */) -> { /* fill in */ }
    // etc., add other common logic or composition operations as needed.
}
```

BTW, if you are wondering about the `.clone()`s - these are really not necessary, and they result in extra memory allocation. The "correct" way to write the code is to use the standard library's reference-counted pointer type, `Rc`[5] in your language `enum` instead of plain `Box`. Then, instead of using `Box::new()` your shortcut methods would look something like this:

```
impl RandProgP2 {
    pub fn seq(prog1: &Rc<RandProgP2>, prog2: &Rc<RandProgP2>) -> Rc<RandProgP2> {
        Rc::new(RandProgP2::Seq(
            Rc::clone(prog1),
            Rc::clone(prog2)
        ))
    }
}
```

This uses `Rc::clone`, which is a shallow clone that only copies the reference-counted pointer. Although not required, you are welcome to use this alternate design if you would like to. (It should only affect your internal methods.)

[5] See https://doc.rust-lang.org/std/rc/struct.Rc.html.